Sign.com Document ID: a5d0ec12c1 - Page 1/21

Code Inspection Report

From: CollaborAITE

To: AnaToPrint (The Slice is Right)

Course: COS 497 – Capstone II

Team Members: Ethan Morin, Clara Scalzer, Ryan Brown, Saurav Pokhrel, and Michael Ness

Date: March 11th, 2026

AnaToPrint

Code Inspection Report **Table of Contents**

Table of Contents.....2
Executive Summary 3
1. Introduction.....4
1.1 Purpose of This Document.....4
References.....4
1.2 Coding and Commenting Conventions.....5
1.3 Defect Checklist.....5
2. Code Inspection Process.....7
2.1 Description.....8
2.2 Impressions of the Process.....8
2.3 Inspection Meetings.....9
3. Modules Inspected 10
3.1 UploadPage.tsx (webapp/src/pages/UploadPage.tsx).....10
3.2 useDicomUpload.ts (webapp/src/hooks/useDicomUpload.ts).....10
3.3 dicomUtils.ts (webapp/src/utils/dicomUtils.ts).....11
3.4 Viewer3D.tsx (webapp/src/components/Viewer3D.tsx) 11
3.5 ExportPage.tsx (webapp/src/pages/ExportPage.tsx).....11
3.6 stlExport.ts (webapp/src/utils/stlExport.ts).....12
3.7 dicomToPng.ts (webapp/src/utils/dicomToPng.ts).....12
3.8 SettingsPage.tsx (webapp/src/pages/SettingsPage.tsx).....12
3.9 useAppSettings.ts (webapp/src/hooks/useAppSettings.ts) 13
3.10 storage.ts (webapp/src/utils/storage.ts) 13
3.11 ErrorBoundary.tsx (webapp/src/components/ErrorBoundary.tsx) 13
3.12 App.tsx (webapp/src/App.tsx) 13
3.13 types/index.ts (webapp/src/types/index.ts) 13

3.14 Modules Not Yet Completed 14
4. Defects .....14 Appendix A - Coding and Commenting Conventions.....17 Appendix B – Team Review Sign-off .....18 Appendix C – Document Contributions 19

Page

1. Introduction3
1.1 Purpose of This Document..... 3
References..... 3 1.2 Coding and Commenting Conventions..... 3 1.3 Defect Checklist..... 4
2. Code Inspection Process..... 6
2.1 Description..... 6
2.2 Impressions of the Process..... 6 2.3 Inspection Meetings..... 7
3. Modules Inspected ..... 8 3.1 UploadPage.tsx (webapp/src/pages/UploadPage.tsx)..... 8 3.2 useDicomUpload.ts (webapp/src/hooks/useDicomUpload.ts)..... 8 3.3 dicomUtils.ts (webapp/src/utils/dicomUtils.ts)..... 9 3.4 Viewer3D.tsx (webapp/src/components/Viewer3D.tsx) ..... 9 3.5 ExportPage.tsx (webapp/src/pages/ExportPage.tsx)..... 9
3.6 stlExport.ts (webapp/src/utils/stlExport.ts)..... 10
3.7 dicomToPng.ts (webapp/src/utils/dicomToPng.ts)..... 10
3.8 SettingsPage.tsx (webapp/src/pages/SettingsPage.tsx)..... 10 3.9 useAppSettings.ts (webapp/src/hooks/useAppSettings.ts) ..... 10 3.10 storage.ts (webapp/src/utils/storage.ts) ..... 11 3.11 ErrorBoundary.tsx (webapp/src/components/ErrorBoundary.tsx) ..... 11 3.12 App.tsx (webapp/src/App.tsx) ..... 11 3.13 types/index.ts (webapp/src/types/index.ts) 11
3.14 Modules Not Yet Completed 11
4. Defects ..... 12 Appendix A - Coding and Commenting Conventions.....

16 Appendix B – Team Review Sign-off .....
17 Appendix C – Document Contributions 18

Executive Summary

CollaborAITE conducted a code inspection of AnaToPrint, a client-side web application that converts medical CT scans in DICOM format into 3D-printable outputs. The inspection focused on correctness, privacy, performance, usability, maintainability, and alignment with the project documentation. The review combined independent pre-inspection analysis with a live inspection meeting involving both teams.

The most significant confirmed defect is the inclusion of protected health information (PHI) in exported ZIP files generated by the PNG export workflow. Additional high-prirority issues include main-thread blocking during STL generation, unhandled error paths, and a GPU memory leak in the 3D viewer. Beyond confirmed defects, the inspection also identified several design and maintainability concerns, including oversized components, inconsistent color tokens, limited comments, and the need for a clarified deployment security model.

- Critical defect requiring immediate remediation: PHI leakeage in exported ZIP metadata. • High priority defects to address before final delivery: STL export threading and error handling, Viewer3D cleanup, ExportPage timr clean up, and resrouce-leak paths in DICOM processing.
- Requirements-driven gaps are still visible in the current implementation. There is a missing surface/volume toggle, missing FPS indicator, and missing file-size validation tied to the UIDD.
- Process recommendations: track confirmed defects separately from architectural recommendations so the receiving team can prioritize fixes more effectively.

1. Introduction

AnaToPrint is a client-side web application developed by The Slice is Right (Israk Arafat, Gregory Michaud, Cooper Stepenkiw, Bryan Sturdivant, and Ethan Wyman) for the Laboratory for Convergent Science at the University of Maine, under the supervision of Dr. Terry Yoo. The application converts medical CT scans in DICOM format into 3D-printable files (STL and G code), with all processing performed entirely in the user's browser to protect patient privacy. The project is built with React, TypeScript, Vite, VTK.js for 3D visualization, and ITK-WASM for DICOM processing. Our team, CollaborAITE (Ethan Morin, Clara Scalzer, Ryan Brown, Saurav Pokhrel, and Michael Ness), conducted a thorough code inspection of the AnaToPrint codebase.

1.1 Purpose of This Document

This document formally records CollaborAITE's code inspection activities performed on The Slice is Right's AnaToPrint project. It is intended for the course instructor and client, Dr. Terry Yoo, as well as for the development team receiving the inspection results. The report summarizes the coding standards adopted by The Slice is Right, the inspection processes used by our team, the modules inspected, and the defects and quality concerns identified during both our pre meeting code review and the live inspection meeting held with The Slice is Right team.

References

1. The Slice is Right, “System Requirements Specification,” University of Maine, COS 497, 2025-2026.
2. The Slice is Right, “System Design Document,” University of Maine, COS 497, 2025-2026.
3. The Slice is Right, “User Interface Design Document,” University of Maine, COS 497, 2025-2026.
4. The Slice is Right, “Critical Design Review Document,” University of Maine, COS 497, 2025-2026.

1.2 Coding and Commenting Conventions

The Slice is Right's codebase is written primarily in TypeScript using the React framework, with Vite as the build tool and Tailwind CSS for styling. The project follows a component-based architecture with pages (pages/), reusable components (components/), custom hooks (hooks/), utility modules (utils/), a shared context (contexts/), and a centralized types file (types/index.ts). The team uses strictly typed TypeScript throughout most of the codebase, with explicit interfaces for props, state, and utility function parameters (e.g., STLExportOptions, DicomFileInfo, AppSettings). Naming conventions follow standard React/TypeScript patterns: PascalCase for components and types, camelCase for functions and variables, and UPPER_SNAKE_CASE for constants (e.g., HU_THRESHOLDS).

Commenting in the codebase is limited. Utility functions in stlExport.ts and dicomUtils.ts include JSDoc-style comments describing parameters and return types, which is good practice. However, the React components and hooks are largely uncommented, with only occasional inline comments. Given that a significant portion of the code was committed by a single team member (~15k lines), the lack of comments raises maintainability concerns. The team does not appear to have adopted a formal commenting convention document. During our meeting, the CollaborAITE advised the Slice is Right to add more comments, particularly for complex logic and in files authored primarily by one developer.

1.3 Defect Checklist

The following checklist was used during our inspection process to identify defects:

#	Category	Defect Type	Description
1	Coding Conventions	‘any’ type usage	TypeScript ‘any’ casts that bypass type safety
2	Coding Conventions	Inconsistent color tokens	Mixing of color values without a central theme/token system
3	Coding Conventions	Oversized components	Components mixing business logic, state management, and UI
4	Coding	Misplaced dependencies	Dependencies declared in wrong

	Conventions		‘package.json’ within the monorepo

5	Logic Errors	Missing error handling (try/catch)	Async operations not wrapped in try/catch blocks
6	Logic Errors	Race conditions / timer leaks	‘setTimeout’ callbacks without cleanup on unmount
7	Logic Errors	Incorrect try/catch scope	Try/catch wrapping ‘setTimeout’ call but not its callback
8	Logic Errors	Patient ID collision	‘groupDicomFiles’ merging patients with missing IDs under “unknown”
9	Logic Errors	Unhandled promise rejections	Promises that can hang or silently fail
10	Security / Privacy	PHI leakage in exports	Protected health information written to metadata.json in zip exports
11	Security / Privacy	PHI displayed in UI	Patient name, ID, DOB, sex rendered directly without minimization
12	Security / Privacy	User-controlled filenames in downloads	Unsanitized filenames used directly in download links
13	Security / Privacy	Raw error messages shown to user	ErrorBoundary diplays ‘error.toString()’ in the UI
14	Security / Privacy	Console logging of implementation details	‘console.error’/’console.warn’ calls leaking internal data to browser
15	Security / Privacy	Known CVEs in dependency tree	Vulnerabilities revealed by ‘npm’ and ‘audit’
16	Security / Privacy	No user authentication	No authentication features implemented
17	Performance	Main-thread blocking (STL export)	Marching cubes + smoothing + writing runs on main thread via ‘setTimeout(, 0)’
18	Performance	GPU memory leak	Viewer3D cleanup does not ‘.delete()’ mapper, volume, actor, or transfer functions
19	Performance	Resource leak on ITK WASM failure	Web worker pool not terminated if ‘readImageDicomFileSeries()’ throws
20	Performance	No file size validation	No upload size limit enforced;

			browser memory is the only constraint
21	Commenting / Documentation	Insufficient Comments	Most components and hooks lack comments; single-developer code is especially under-documented
22	Commenting / Documentation	Misleading Save Settings button	Button calls ‘alert()’ only; actual saving happens automatically via ‘updateSettings’
23	UI / Usability	DICOM file picker broken	File explorer does not show DICOM files; only drag-and-drop works
24	UI / Usability	No drag-and-drop file type validation	Non-DICOM files accepted via drag-and drop without validation
25	UI / Usability	Missing Surface/Volume render toggle	UIDD DV-04 specifies a render mode toggle; only volume rendering exists
26	UI / Usability	No FPS indicator in status bar	UIDD 3.3 specifies FPS in status bar; not implemented
27	UI / Usability	Settings use sessionStorage	Settings lost on tab/window close; comment on line 3 of storage.ts suggests localStorage was intended
28	UI / Usability	Camera reset overwrites user position	‘handleReset’ in Viewer3D.tsx sets hardcoded position/viewUp/focalPoint after ‘resetCamera()’
29	Other	Permanent loading state on conversion failure	‘useDicomUpload’ can get stuck if ‘convertItkToVtkImage()’ fails between state updates
30	Other	No WebWorkerManager for STL generation	SDD Section 2.2 specifies WebWorkerManager for marching cubes offloading; not implemented

2. Code Inspection Process

The code inspection was conducted after The Slice is Right's critical design review and during their mid-to-late implementation phase. The team's codebase compiles and builds without fatal errors. The project has both unit tests and end-to-end tests. Our goal was to identify defects spanning correctness, security, performance, coding conventions, and usability before the project moves further into testing and deployment.

2.1 Description

Our team's inspection process consisted of two phases. In the first phase, two selected members of CollaborAITE independently cloned the AnaToPrint repository, built the application, and reviewed assigned modules. We compiled a shared document of pre-meeting notes, questions, and suspected defects. We cross-referenced the code against The Slice is Right's own SRS, SDD, and UIDD to identify requirement deviations and specification gaps.

In the second phase, we held a live inspection meeting with The Slice is Right. During this meeting, we walked through our pre-meeting findings module by module, asked clarifying questions, and recorded their responses. This two-phase approach allowed us to cover the codebase thoroughly while also getting the implementing team's perspective on design decisions and known limitations.

The team did have a designated notetaker, moderator, inspector, and reader. However, we used a more conversational approach during the meeting, which allowed for deeper discussion of complex issues (such as the PHI leakage and main-thread blocking concerns) but sometimes led to tangential discussion. Despite this, the process was effective at surfacing a substantial number of defects across all categories.

2.2 Impressions of the Process

The code inspection was highly effective at identifying defects that would have been costly to discover during testing or after deployment. The most impactful finding was the PHI leakage in exported zip files (metadata.json containing patientID, patientName, patientDateOfBirth, and patientSex), which directly violates NFR-05 and could have serious privacy implications for a medical application. This type of defect is difficult to catch through automated testing alone and demonstrates the value of manual code inspection. The security-related findings (unsanitized filenames, raw error messages, console logging, known CVEs) were also surfaced effectively through this process.

The strongest module reviewed in the codebase, in our assessment, is 'dicomUtils.ts'. It demonstrates good practices: proper use of web workers for DICOM tag parsing, chunked processing to avoid overwhelming the system, structured error handling within the parse loop, and clean TypeScript interfaces. The 'stlExport.ts' utility is also well-structured with clear

interfaces, JSDoc comments, and a logical pipeline, though it has the significant main-thread blocking issue.

The modules with the greatest concentration of defects are 'ExportPage.tsx' and 'UploadPage.tsx'. Both are oversized components that mix business logic, state management, file traversal, and UI rendering into a single file. 'ExportPage.tsx' contains the race condition with the 1.2-second timer (lines 112-114), inconsistent error handling between the STL and G-code export paths, and no cleanup on unmount. 'UploadPage.tsx' has the unhandled promise in its file tree traversal (lines 94-96), heavy use of any types in the 'traverseFileTree' function, and no file type validation on drag and-drop. These components would benefit significantly from being decomposed into smaller, more focused units with separated concerns.

2.3 Inspection Meetings

Meeting 1: Pre-Inspection Preparation (CollaborAITE internal)

- **Date:** February 27, 2026
- **Location:** Virtual (Zoom)
- **Time:** 13:00-13:50
- **Participants:** Ethan Morin, Clara Scalzer, Ryan Brown, Saurav Pokhrel, Michael Ness
- Modules Covered: Modules were discussed among team members for independent review. Pre-meeting notes and questions were compiled into a shared document.

Meeting 2: Live Code Inspection with The Slice is Right

- **Date:** March 4, 2026
- **Location:** FEEDC 236
- **Time:** 14:15-14:45
- **Participants (CollaborAITE):** Ethan Morin, Clara Scalzer, Ryan Brown, Saurav Pokhrel, Michael Ness
- **Participants (The Slice is Right):** Israk Arafat, Gregory Michaud, Cooper Stepenkiw, Bryan Sturdivant, Ethan Wyman
- **Roles:**
  - **Moderator**: Ethan Morin
  - **Scribe:** Clara Scalzer
  - **Inspector:** Michael Ness
  - **Reader:** Saurav Pokhrel, Ryan Brown
- **Modules Covered:** UploadPage.tsx, useDicomUpload.ts, dicomUtils.ts, Viewer3D.tsx, ExportPage.tsx, stlExport.ts, dicomToPng.ts, SettingsPage.tsx, useAppSettings.ts, storage.ts, ErrorBoundary.tsx, App.tsx, types/index.ts
- **Key Discussion Points:**
  - Upload flow: File explorer does not display DICOM files; only drag-and-drop works. Team confirmed they expect folder uploads only, but drag-and-drop still accepts unchecked files.
  - FPS monitoring: Team confirmed no embedded FPS display; they assess smoothness visually (~15 FPS observed) and plan to implement testing.
  - Settings page: Team acknowledged it is minimal and considered unnecessary for

now.
  - Code ownership: One developer committed ~15k lines. Team relies on branch protections and CI workflows; no peer approval required for merges.
  - Viewer3D cleanup: Team acknowledged the GPU memory leak and is looking into it.
  - Surface rendering: Does not exist; only volume rendering is implemented. Team is aware.
  - npm audit: Advised to run; multiple CVEs expected.
  - Try/catch gaps: Advised on multiple unguarded async calls. Team is looking into the ExportPage timer issue (lines 112-114).
  - Code style: ExportPage and UploadPage identified as oversized. No central color token repository. More comments needed.

3. Modules Inspected

Section 3 describes the modules selected for inspection and explains why each was included in the review. These modules were chosen because they represent important functional areas of the system and help evaluate the application's overall quality, maintainability, and alignment with the project's documented requirements.

3.1 UploadPage.tsx (webapp/src/pages/UploadPage.tsx)

- **Functionality:** Main upload interface for DICOM files. Supports drag-and-drop folder upload and file browser selection. Traverses dropped folder structures, triggers DICOM parsing via useDicomUpload, displays series selection when multiple series are found, and navigates to preview or export on completion.
- **SDD Mapping:** Implements the Upload Module / File Input component from the SDD. • **Deviations From SDD:**
  - The file browser (webkitdirectory) does not show individual DICOM files, only folders. The SRS implies users should be able to select DICOM files.
  - No file size validation is implemented. The UIDD specifies a max 1.0 GB file size limit.
  - No file type validation on drag-and-drop. Non-DICOM files are accepted and passed to the parser, wasting CPU.
  - The traverseFileTree function uses any types extensively, bypassing TypeScript's type safety.

3.2 useDicomUpload.ts (webapp/src/hooks/useDicomUpload.ts)

- **Functionality:** Custom React hook managing the DICOM upload lifecycle: parsing files, grouping by series, loading selected series into ITK image data, and converting to VTK format.
- **SDD Mapping:** Part of the Upload Module / DICOM Processing pipeline.
- **Deviations From SDD:**
  - loadDicomImageSeries() and convertItkToVtkImage() (lines 65-76) are called without individual try/catch. If conversion fails between state updates, the UI can get stuck in a permanent loading state.
  - The vtkImage state is typed as any (line 26), losing type safety for a critical data structure.

3.3 dicomUtils.ts (webapp/src/utils/dicomUtils.ts)

• **Functionality:** Core DICOM processing utilities: parsing DICOM tags via ITK-WASM web workers, loading image series, and grouping files by patient/study/series. • **SDD Mapping:** Implements the DICOM Parser and File Grouping modules from the SDD.

• **Deviations From SDD:**

- readImageDicomFileSeries() (lines 154-160) is called without try/catch. If the ITK-WASM pipeline fails, the web worker pool is never terminated (resource leak).
- groupDicomFiles uses "unknown" as a fallback for missing patient IDs, causing unrelated patients' scans to be merged into the same bucket.

3.4 Viewer3D.tsx (webapp/src/components/Viewer3D.tsx)

• **Functionality:** 3D volume renderer using VTK.js. Creates a full-screen render window, volume mapper, and actor with configurable color/opacity transfer functions. Supports camera reset and window/level adjustments.

• **SDD Mapping:** Implements the 3D Viewer / Volume Rendering component. • **Deviations From SDD:**

- The cleanup function (lines 122-136) only calls fullScreenRendererRef.current.delete(). The mapper, volume actor (vtkVolume), color transfer function, and opacity function are never .delete()'d, leaking GPU memory every time the component remounts.
- UIDD DV-04 specifies a surface/volume render mode toggle. No render mode selector exists; only volume rendering is implemented.
- handleReset (lines 179-192) calls resetCamera() then immediately overwrites with hardcoded setPosition(0, 0, 1), setViewUp(0, 1, 0), and setFocalPoint(0, 0, 0), then calls resetCamera() again. This makes the first resetCamera() call pointless and may produce unexpected camera behavior.

3.5 ExportPage.tsx (webapp/src/pages/ExportPage.tsx)

• **Functionality:** Export interface allowing users to choose between STL and G-code (PNG zip) export formats. Configures threshold, smoothing, and filename. Displays progress via ExportProgressModal.

• **SDD Mapping:** Implements the Export Module.

• **Deviations From SDD:**

- Race condition at lines 112-114: a 1.2-second setTimeout fires unconditionally after G-code export. If the user navigates away before it fires, setState is called on an unmounted component. Rapid clicks queue multiple timers. No clearTimeout in any cleanup path.
- The STL export path (outer try/catch) will not catch errors thrown inside generateSTL because stlExport.ts's exportToSTL wraps the call in setTimeout(, 0) whose callback is outside the try/catch scope.
- The G-code path has an inner try/catch with alert and state reset, but the STL path has no equivalent inner error handling.
- User-controlled filename is used directly in download links without sanitization.

3.6 stlExport.ts (webapp/src/utils/stlExport.ts)

• **Functionality:** Generates STL files from VTK image data using marching cubes surface extraction, optional windowed sinc smoothing, and VTK STL writer. Provides download functionality.

• **SDD Mapping:** Implements the STL Export / Marching Cubes pipeline from the SDD. • **Deviations From SDD:**

- SDD Section 2.2 specifies a WebWorkerManager to offload marching cubes to avoid blocking the UI. Currently, exportToSTL wraps the entire pipeline in setTimeout(, 0), which still runs on the main thread and will freeze the UI for large datasets.
- NFR-08 requires main-thread Long Tasks < 200ms and interaction FPS >= 20 during orbit/zoom. The current setup cannot meet this for non-trivial datasets.
- The try/catch in exportToSTL (lines 165-185) wraps only the setTimeout() call itself, not the callback. Any exception inside the callback (lines 167-181) will result in an unhandled promise rejection.

3.7 dicomToPng.ts (webapp/src/utils/dicomToPng.ts)

- **Functionality:** Converts DICOM files to PNG images, organized by series. Supports windowing, scaling, and zip export with metadata
- **SDD Mapping:** Part of the G-code Export pipeline (PNG intermediate step).
- **Deviations From SDD:**
  - **Critical (NFR-05 violation):** downloadOrganizedPngs (lines 266-270) writes a metadata.json into the zip containing patientID, patientName, patientDateOfBirth, and patientSex. NFR-05 states "exported files must not contain embedded protected health information." The user is not informed of this data inclusion.
  - zip.generateAsync() at line 277 has no .catch(); a corrupted zip silently fails.
  - User-controlled zipFileName is used directly in the download link without sanitization.

3.8 SettingsPage.tsx (webapp/src/pages/SettingsPage.tsx)

- **Functionality:** Displays application settings (render quality, background color, grid toggle, auto-optimize) and a quick start guide.
- **SDD Mapping:** Implements the Settings / Help page.
- **Deviations From SDD:**
  - The "Save Settings" button (line 6-8) only calls alert("Settings saved successfully!"). Actual persistence happens automatically via updateSettings on every change. The button is misleading.
  - Most settings are disabled or have only a single option (render quality: only "High"; background color: only "White"; grid: disabled checkbox; auto-optimize: disabled checkbox). The settings page is largely non-functional.

3.9 useAppSettings.ts (webapp/src/hooks/useAppSettings.ts)

- **Functionality:** Custom hook for loading and saving application settings.
- **SDD Mapping:** Part of the Settings Module.
- **Deviations From SDD:**
  - No significant deviations; functions as a thin wrapper around storage.ts.

3.10 storage.ts (webapp/src/utils/storage.ts)

- **Functionality:** Provides sessionStorage-based persistence for application settings. • **SDD Mapping:** Part of the Settings / Persistence layer.
- **Deviations From SDD:**
  - Uses sessionStorage instead of localStorage. Settings are lost when the tab/window is closed. The comment on line 7 ("Save data to session storage") and the naming suggest this may be intentional, but the SettingsPage comment on line 7 ("We will save them in local storage or cookie") suggests localStorage was the

intended target.

3.11 ErrorBoundary.tsx (webapp/src/components/ErrorBoundary.tsx)

- **Functionality:** React error boundary that catches unhandled errors and displays a fallback UI with a reload button.
- **SDD Mapping:** Part of the Error Handling / UI Shell.
- **Deviations From SDD:**
  - Displays the raw error message (this.state.error.toString()) directly in the UI (line 34). This can expose implementation details, stack traces, or sensitive information to end users.

3.12 App.tsx (webapp/src/App.tsx)

- **Functionality:** Root application component. Sets up React Router, context providers, error boundary, and lazy-loaded page routes.
- **SDD Mapping:** Implements the application shell and routing architecture.
- **Deviations From SDD:**
  - No significant deviations from the SDD. Uses lazy loading with Suspense for code splitting, which is a good practice.

3.13 types/index.ts (webapp/src/types/index.ts)

- **Functionality:** Centralized TypeScript type definitions for the application.
- **SDD Mapping:** Shared types layer.
- **Deviations From SDD:**
  - No significant deviations. Types are well-defined and used throughout the codebase.

3.14 Modules Not Yet Completed

- **WebWorkerManager for STL generation:** SDD Section 2.2 specifies this module to offload marching cubes processing. Not yet implemented. The team should target completion to meet NFR-08 performance requirements.
- **Surface rendering mode:** UIDD DV-04 specifies a surface/volume render toggle. Only volume rendering exists. The team is aware.
- **FPS monitoring / status bar display:** UIDD 3.3 specifies FPS in the status bar. The team plans to implement testing for this.

4. Defects

The table below lists the defects found during the code inspection.

#	Module	Defect Description	Category	Severity

1	dicomToPng.ts (lines 266-270)	downloadOrganizedPngs writes metadata.json into the export zip containing patientID, patientName, patientDateOfBirth, and patientSex. This violates NFR-05: "exported files must not contain embedded protected health information." The user is not informed this data is included.	Security / Privacy	Critical
2	UploadPage.tsx (lines 269-288)	Patient name, ID, DOB, sex, study date, and series description are parsed from DICOM tags and rendered directly in the UI without any PHI minimization or disclosure notice.	Security / Privacy	High
3	stlExport.ts (lines 164-186)	exportToSTL wraps generateSTL in setTimeout(, 0). The try/catch (line 165) only wraps the setTimeout() call itself, not the callback. Any exception thrown inside the callback (marching cubes, smoothing, writing) will not be caught. The catch block in ExportPage.tsx will never fire for these errors, leaving the UI in a processing state with no visible error.	Logic Error	High
4	stlExport.ts (lines 164-186)	The entire marching cubes + smoothing + STL writing pipeline runs on the main thread via setTimeout(, 0). SDD Section 2.2 specifies a WebWorkerManager for this. Large DICOM datasets will freeze the UI. Violates NFR-08 (main-thread Long Tasks < 200ms).	Perfor man ce	High
5	Viewer3D.tsx (lines 122-136)	Cleanup function only calls fullScreenRendererRef.current.delete(). The VTK mapper, volume actor (vtkVolume), color transfer function, and opacity function are never .delete()'d. This leaks GPU memory every time Viewer3D is remounted.	Perfor man ce	High
6	ExportPage.tsx (lines 112-114)	A 1.2-second setTimeout fires unconditionally after G-code export completion. If the user navigates away before it fires, setState is called on an unmounted component. Rapid clicks queue multiple timers. No clearTimeout exists in any cleanup path.	Logic Error	High
7	dicomUtils.ts (lines 154-160)	readImageDicomFileSeries() is called without try/catch. If the ITK-WASM pipeline fails, the webWorkerPool is never terminated, causing a resource leak.	Logic Error	High

8	UploadPage.tsx (lines 94-96)	The traverseFileTree helper uses entry.file() with a callback wrapped in a Promise, but the Promise has no reject path. If entry.file() fails, the Promise hangs forever.	Logic Error	Medium
9	useDicomUpload.ts (lines 65-76)	loadDicomImageSeries() and convertItkToVtkImage() are called inside loadSeries without individual try/catch. If the conversion fails between state updates (e.g., after isLoading is set but before isComplete), the UI can get stuck in a permanent loading state.	Logic Error	Medium
10	dicomToPng.ts (line 277)	zip.generateAsync({ type: "blob" }) has no .catch() handler. A corrupted zip operation will silently fail with no user feedback.	Logic Error	Medium
11	dicomUtils.ts (lines 185-187)	groupDicomFiles uses "unknown" as the fallback for missing patientID, studyInstanceID, and seriesInstanceID. If two different patients both have missing IDs, their scans are merged into the same bucket and treated as one patient.	Logic Error	Medium
12	ErrorBoundary.tsx (lines 33-36)	When the application crashes, the raw error message (this.state.error.toString()) is displayed directly in the UI via a <pre> tag. This can expose stack traces, file paths, and implementation details to end users.	Security / Privacy	Medium
13	stlExport.ts (line 124), dicomToPng.ts (lines 282-283)	User-controlled filenames are used directly in download link href/download attributes without sanitization. Malicious filenames could potentially cause unexpected behavior.	Security / Privacy	Medium
14	Multiple files	console.error and console.warn calls throughout the codebase (dicomUtils.ts:97, dicomToPng.ts:192/241, useDicomUpload.ts:89, ExportPage.tsx:116/127, ErrorBoundary.tsx:22) leak implementation details, error objects, and stack traces to the browser console.	Security / Privacy	Low
15	Dependency tree	Running npm audit reveals multiple known CVEs in the project's dependency tree. No evidence of regular audit or remediation.	Security / Privacy	Medium

16	UploadPage.tsx	No file type validation on drag-and-drop. Any file type is accepted and passed to the DICOM parser, wasting CPU on files that will never parse successfully.	UI / Usability	Medium
17	UploadPage.tsx	The file browser uses webkitdirectory, which only allows folder selection. DICOM files cannot be individually selected from the file explorer. Only drag-and-drop of folders works for upload.	UI / Usability	Medium
18	Upload flow	No file size validation exists anywhere in the upload pipeline. The UIDD specifies a max 1.0 GB file size limit. Currently, the only limit is the browser's available memory.	Correctness	Medium
19	Viewer3D.tsx	UIDD DV-04 specifies a surface/volume render mode toggle. No render mode selector exists in the UI. Only volume rendering is implemented.	Correctness	Medium
20	Application wide	UIDD 3.3 specifies FPS display in the status bar. No FPS monitoring or display is implemented. The team observed ~15 FPS visually but has no embedded measurement.	Correctness	Medium
21	SettingsPage.tsx (lines 6-8)	The "Save Settings" button only calls alert("Settings saved successfully!"). Actual saving happens automatically through updateSettings on every change. If a user changes settings without clicking the button, they are still saved. The button is misleading.	UI / Usability	Low
22	storage.ts	Settings persistence uses sessionStorage rather than localStorage. Settings are lost when the browser tab or window is closed. The comment in SettingsPage.tsx line 7 ("We will save them in local storage or cookie") suggests localStorage was the intended implementation.	Logic Error	Low
23	ExportPage.tsx	The G-code export path (lines 85-120) has an inner try/catch that calls alert and resets state on error. The STL export path has no equivalent inner error handling -- errors in generateSTL are silently swallowed due to D-03. Inconsistent error handling between export paths.	Logic Error	Medium

24	Viewer3D.tsx (lines 179-192)	handleReset calls renderer.resetCamera(), then immediately overwrites with hardcoded camera.setPosition(0, 0, 1), camera.setViewUp(0, 1, 0), camera.setFocalPoint(0, 0, 0), then calls renderer.resetCamera() again. The first resetCamera() is rendered pointless; the hardcoded values override any data-aware camera positioning.	Logic Error	Low
25	package.json (root)	The jszip and @types/jszip dependencies are declared in the root package.json rather than in webapp/package.json, which is the package that actually imports them. If the webapp were extracted from the monorepo, it would break.	Coding Convention	Low
26	ExportPage.tsx, UploadPage.tsx	Both are oversized components mixing business logic, state management, file traversal, and UI rendering. This reduces testability, readability, and maintainability. Should be decomposed into smaller units with separated concerns.	Coding Convention	Medium
27	Multiple files	TypeScript any casts are used at important points: useDicomUpload.ts line 26 (vtkImage: any), UploadPage.tsx line 92 (entry: any), SettingsPage.tsx line 63 (as any). These bypass TypeScript's type safety at critical junctures.	Coding Convention	Low
28	Application wide	Inconsistent color tokens across the site. Tailwind utility classes use ad-hoc color values (e.g., bg blue-500, bg-indigo-500, bg-emerald-500, bg green-600) with no central color theme or design token system.	Coding Convention	Low
29	Multiple components	Insufficient code comments throughout the codebase. React components and hooks are largely uncommented. Given that ~15k lines were committed by a single developer, this is a significant maintainability risk.	Coding Convention	Medium
30	Application wide	No user authentication is implemented. Depending on the deployment scenario, this may or may not be a requirement, but it is worth noting for a medical application handling PHI.	Security / Privacy	Low

Appendix A - Coding and Commenting

- **TypeScript** (strict mode) with **React 19** for UI components
- **Vite 7** as the build tool
- **Tailwind CSS 4** for styling
- **VTK.js** for 3D volume rendering
- **ITK-WASM** for DICOM file parsing and image processing
- **Vitest** for unit/component tests; **Playwright** for end-to-end tests

Naming Conventions Observed

- **Components:** PascalCase (e.g., UploadPage, Viewer3D, ExportProgressModal) •

Hooks: camelCase with use prefix (e.g., useDicomUpload, useAppSettings)

- **Utility functions:** camelCase
  (e.g., parseDicomFiles, generateSTL, downloadOrganizedPngs)
- **Interfaces/Types:** PascalCase (e.g., DicomFileInfo, STLExportOptions, AppSettings)
- **Constants:** UPPER_SNAKE_CASE
  (e.g., HU_THRESHOLDS, DEFAULT_SETTINGS, STORAGE_PREFIX)
- **Files:** PascalCase for components/pages, camelCase for utilities/hooks

Code Style

- Consistent use of const for functional components and arrow functions •

React hooks (useState, useEffect, useCallback, useRef) used throughout

- Tailwind utility classes for all styling (no separate CSS files observed) •
Lazy loading with React.lazy() and Suspense for page-level code splitting
- Monorepo structure with npm workspaces (webapp workspace)

Commenting Conventions Observed

- JSDoc-style comments on exported utility functions (good, but inconsistent) • Inline comments used sparingly, mostly for section headers (e.g., // Cleanup, // Create marching cubes filter)
- No file-level header comments
- No formal commenting standard documented by the team
- Components and hooks are largely uncommented

Areas for Improvement

- Adopt a formal JSDoc standard for all exported functions and interfaces •
Add file-level documentation describing purpose and dependencies
- Add inline comments for non-obvious logic (especially in traverseFileTree, export flows, and VTK setup)
- Establish a central color/design token system instead of ad-hoc Tailwind classes •
Reduce any usage; define proper types for VTK objects and file system API entries

Appendix B – Team Review Sign-off

All members of CollaborAITE have reviewed this Code Inspection Report and agree on its content and format.

Name	**Signature**	**Date**	**Comments**
Ethan Morin	Ethan Morin	11 MARCH 2026	
Clara Scalzer		11 March 2026	
Ryan Brown	Ryan Brown	11 March 2026	
Saurav Pokhrel		11 March 2026	
Michael Ness		11 March 2026	

Sign.com Document ID: a5d0ec12c1 - Page 21/21

Appendix C – Document Contributions

Name	Sections Contributed	% Contributed
Ethan Morin	Inspection preparation, live review, document review	19%
Clara Scalzer	Inspection preparation, documentation during inspection, live review, CIR draft and review	24%
Ryan Brown	Inspection preparation, live review, document review	19%
Saurav Pokhrel	Inspection preparation, live review, document	19%

	review	
Michael Ness	Inspection preparation, live review, document review	19%
Total:		%100